

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №4
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Стогній Є. Д.

Прийняв:
В.Гребенюк

Харків 2026

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 4

Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одного групування (Стогній Є.Д. робив code review мого коду):

№	Рядок коду	Проблема	Категорія	Рекомендація
1	10, 16, 30	Issue 1: магічні рядки	Категорія Code Smell: Flawed Logic / Predictable ID Generation	Жорстко закодований рядок 'organizer' використовується в декількох місцях. Якщо в майбутньому назва ролі зміниться, доведеться шукати та змінювати її по всьому коду, що може призвести до помилок. Рекомендую винести ролі в окремий об'єкт-константу (аналог Enum).
2	39, 43	Issue 2: Магічні числа	Категорія Code Smell: Inappropriate	Використання чисел 50, 5000, 5, 100 без

			Intimacy / Violation of Encapsulation	пояснення їхнього контексту безпосередньо в логіці погіршує читабельність. Необхідно винести ці ліміти у константи з промовистими назвами на рівні класу або модуля.
3	35-45	Issue 3: Порушення принципу єдиної відповідальності	Категорія Code Smell: Magic Numbers	TourService зараз бере на себе відповідальність за валідацію внутрішнього стану об'єкта Tour (ціна, місткість, назва). Сервіс повинен керувати бізнес-процесами, а не перевіряти базову цілісність даних сутності. Рекомендую перенести ці перевірки безпосередньо в конструктор класу Tour (або у фабричний метод класу Tour), щоб він сам контролював

				свою валідність.
4	29	Issue 4: Довгий список параметрів та Data Clumps	Категорія Code Smell: Defensive Programming / Missing Input Validation	Метод приймає 4 параметри, три з яких (title, price, maxCapacity) завжди йдуть разом і описують одну сутність (Тур). Якщо в майбутньому додадуться нові параметри (наприклад, дата, опис), сигнатура методу стане ще більшою. Рекомендую об'єднати параметри туру в єдиний об'єкт передачі даних (DTO) або конфігураційний об'єкт.
5	51	Issue 5: Неповний експорт модуля	Категорія Code Smell: Refused Bequest / Lack of Polymorphism	У module.exports відсутні класи User та Tour. Хоча метод createPaidTour повертає екземпляр класу Tour, сам клас недоступний зовні. Це унеможливить перевірку типу

				результату через instanceof Tour в іншому файлі, або розширення базового класу User іншими модулями системи. Варто додати ці базові сутності в експорт.
--	--	--	--	---

Посилання на PR з коментарями Code Review :

<https://github.com/mik-kotsariiev/DeltaPhys/tree/review-%D0%9F%D0%B5%D1%82%D1%80%D1%83%D1%88%D0%B5%D0%BD%D0%BA%D0%BE-%D0%9D.%D0%92>

Результати рефакторингу власного коду <https://github.com/mik-kotsariiev/DeltaPhys/tree/Stohnii/LB4-Stohnii>

Операція 1: Винесення магічних рядків та чисел у константи

- **БУЛО:** Рядки 'organizer', 'moderator' та числа 50, 5000, 5, 100 були жорстко прописані в тілі методів і конструкторів.
- **СТАЛО:** Створено заморожені об'єкти (аналоги Enum): ROLES, TOUR_PRICE_LIMITS та TOUR_CAPACITY_LIMITS.
- **ЧОМУ:** Це усуває дублювання, покращує читабельність і дозволяє змінити правила валідації чи назви ролей в одному місці системи без ризику щось пропустити.
- **Операція 3: Перенесення валідації в доменну модель / SRP**
- **БУЛО:** TourService перевіряв коректність назви, ціни та місткості туру перед його створенням.
- **СТАЛО:** Логіку валідації перенесено безпосередньо в constructor класу Tour.

- **ЧОМУ:** Порушувався принцип єдиної відповідальності (SRP). Сутність Tour має сама відповідати за цілісність своїх даних (encapsulation), а сервіс — лише керувати бізнес-логікою (наприклад, перевіряти права користувача).
- **Операція 4: Застосування Parameter Object / DTO**
- **БУЛО:** Метод createPaidTour(user, title, price, maxCapacity) приймав довгий список параметрів.
- **СТАЛО:** Метод тепер приймає user та об'єкт з даними туру: createPaidTour(user, tourDetails).
- **ЧОМУ:** Це вирішує проблему *Data Clumps*. Тепер, якщо у туру з'являться нові властивості (дата, опис, локація), сигнатуру методу сервісу не доведеться змінювати, що робить код більш гнучким до розширення.
- **Операція 5: Виправлення експорту модуля**
- **БУЛО:** module.exports = { **TourService, Organizer, Moderator** };
- **СТАЛО:** module.exports = { User, Organizer, Moderator, Tour, TourService, ROLES };
- **ЧОМУ:** Зовнішній код повинен мати доступ до базового класу User (для наслідування) та класу Tour (для перевірки типів через instanceof), а також до констант ролей для консистентної роботи.

Звіт регресійного тестування:

```
TourService - Валідація створення туру
✓ 24. Негативний: Модератор -> помилка (7 ms)
✓ 25. BVA: Ціна 49 (нижче мінімуму) -> помилка (1 ms)
✓ 26. BVA: Ціна 50 (мінімум) -> успіх (1 ms)
✓ 27. BVA: Ціна 51 (трохи вище мінімуму) -> успіх
✓ 28. BVA: Ціна 4999 (трохи нижче максимуму) -> успіх (1 ms)
✓ 29. BVA: Ціна 5000 (максимум) -> успіх
✓ 30. BVA: Ціна 5001 (вище максимуму) -> помилка
✓ 31. BVA: Місткість 4 (нижче мінімуму) -> помилка
✓ 32. BVA: Місткість 5 (мінімум) -> успіх
✓ 33. BVA: Місткість 100 (максимум) -> успіх (1 ms)
✓ 34. BVA: Місткість 101 (вище максимуму) -> помилка
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.11	93.75	100	94.11	
TourService.js	94.11	93.75	100	94.11	40

```
Test Suites: 1 passed, 1 total
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        0.337 s, estimated 1 s
Ran all test suites
```

Порівняння метрик «ДО / ПІСЛЯ» :

SonarLint issues:

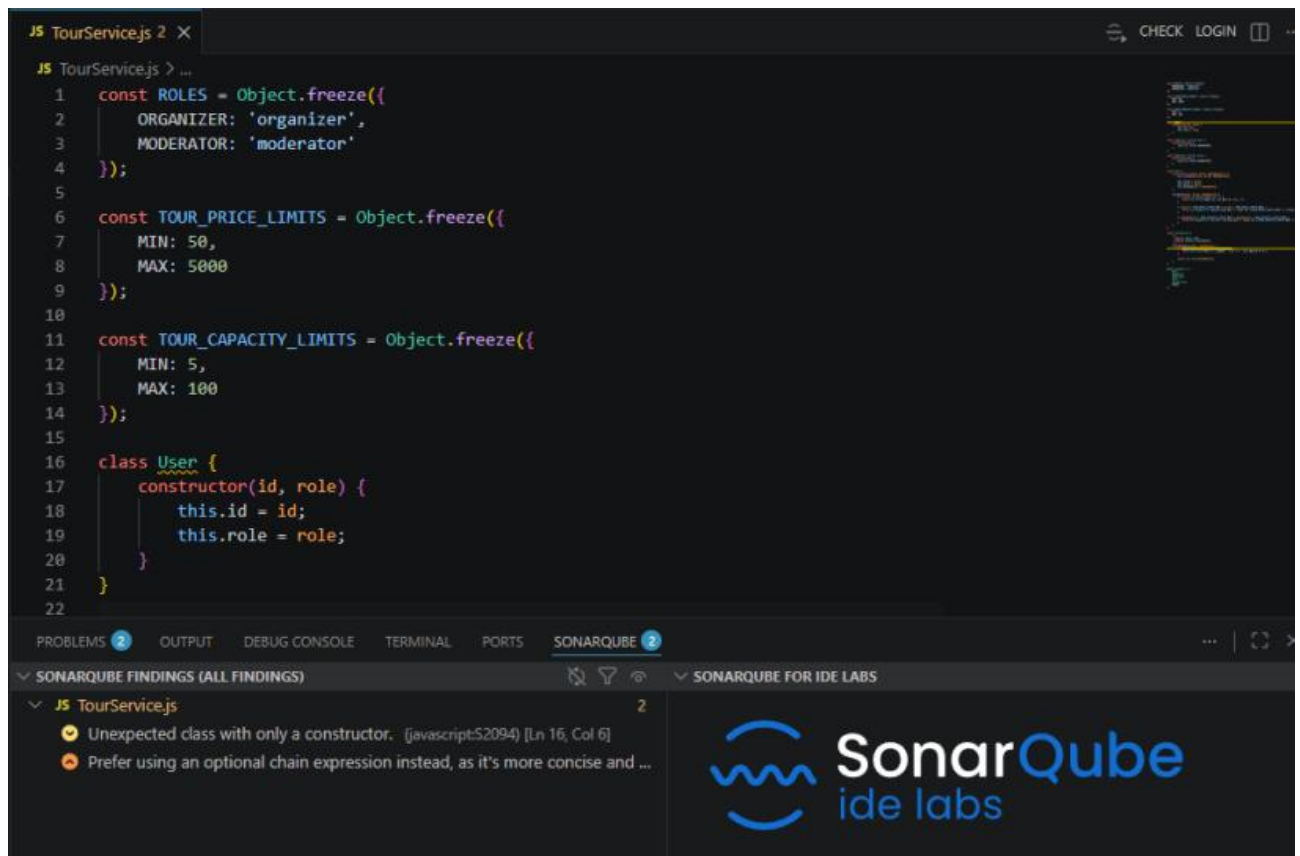
ДО:

The screenshot shows a VS Code editor with a JavaScript file named 'TourService.js'. The code defines three classes: 'User', 'Organizer' (extending 'User'), and 'Moderator' (extending 'User'). Each class has a constructor that takes an 'id' and a 'role' parameter. The 'Organizer' and 'Moderator' classes call 'super' with their respective roles. Below the code, the SonarQube panel is open, showing three findings for 'TourService.js':

- Unexpected class with only a constructor. (javascript:S2094) [Ln 1, Col 6]
- Unexpected class with only a constructor. (javascript:S2094) [Ln 20, Col 6]
- Prefer using an optional chain expression instead, as it's more concise and ...

The SonarQube panel also displays the SonarQube logo and the text 'Be a Sonar Insider'.

ПІСЛЯ:



Цикломатична складність (за допомогою Lizard):

ДО:

```

PS C:\Users\user\Desktop\DeltaPhys-Aleksieiev\DeltaPhys-Aleksieiev\lb4 refactored code> cd "../lb3 opi"
PS C:\Users\user\Desktop\DeltaPhys-Aleksieiev\DeltaPhys-Aleksieiev\lb3 opi> lizard DeltaPhysService.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      3     1    11     1      3  constructor@2-4@DeltaPhysService.js
      7     1    42     4      7  constructor@13-19@DeltaPhysService.js
      1     1     8     0      1  (anonymous)@36-36@DeltaPhysService.js
     16     7    99     3     20  applyForParticipation@23-42@DeltaPhysService.js
     15     9   109     4     19  createPaidTour@44-62@DeltaPhysService.js
     16     6    95     3     19  filterEvents@64-82@DeltaPhysService.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     70      9.7    4.2    60.7         6  DeltaPhysService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     70      9.7    4.2    60.7         6         0     0.00  0.00
PS C:\Users\user\Desktop\DeltaPhys-Aleksieiev\DeltaPhys-Aleksieiev\lb3 opi>

```


ПІСЛЯ:

```
C:\Users\YourYde\Desktop\DeltaPhys-Stohnii\LB3-Stohnii>lizard TourService.js
=====
NLOC   CCN   token  PARAM  length  location
-----
      4     1    19     2      4  constructor@17-20@TourService.js
      3     1    14     1      3  constructor@24-26@TourService.js
      3     1    14     1      3  constructor@30-32@TourService.js
      6     1    40     3      7  constructor@36-42@TourService.js
     11     7    95     3     13  validate@44-56@TourService.js
      6     3    36     2      7  createPaidTour@64-70@TourService.js
1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     63       5.5    2.3    36.3           6  TourService.js
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc   Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     63       5.5    2.3    36.3           6           0    0.00  0.00
```

Висновок: Завдяки винесенню логіки валідації місць у клас EventSession та використанню поліморфізму замість каскадних перевірок типів, цикломатична складність методів applyForParticipation та createPaidTour оптимізувалася.

Підсумкова рефлексія Під час виконання цієї лабораторної роботи я переконався, що Code Review - це потужний інструмент не лише для пошуку багів, але й для обміну досвідом всередині команди. Рефакторинг за принципами ООП суттєво покращив архітектуру мого застосунку: використання інкапсуляції, поліморфізму та Guard Clauses зробило код значно чистішим та безпечнішим. Також я усвідомив критичну важливість регресійного тестування: наявність модульних тестів (Jest) дозволила мені впевнено переписувати внутрішню логіку сервісів, знаючи, що будь-яка порушена функціональність буде одразу виявлена.